



Relazione Progetto Cloud Computing "AI Sport News"



Studente:
Giovanni Lorenzo Murfuni
Mat. 269318

DICEMBRE 2025

Intento

La seguente relazione tecnica descrive le specifiche tecniche e funzionali dell'applicativo "AI Sport News".

L'applicativo implementato consente, ad un generico utente della rete, di visualizzare notizie sportive, le quali vengono automaticamente reperite dal web tramite apposita API.

L'utente può redigere in maniera indipendente notizie sportive di vario tipo, attenendosi al layout consentito dal backend.

A livello tecnico, il backend verrà sviluppato attraverso il framework Django-REST in python con l'integrazione di Celery per gestire tutti i task più rilevanti a livello prestazionale; nello specifico, Celery permette di gestire in maniera asincrona task cruciali per il corretto funzionamento dell'applicativo.

Si è scelto di implementare il tutto in ambiente poetry per semplificare la gestione delle dipendenze e la fase di deploy del database.

Il frontend è stato sviluppato in Flutter; tale framework gode di una discreta possibilità di integrazione con backend basati su API-REST e permette di implementare in maniera semplice (e parallela) applicativi Web e Mobile rendendolo il candidato più promettente per la seguente implementazione.

Indice

1	Backend - Django	3
1.1	Modello Entità Relazione	3
1.1.1	Utente	3
1.1.2	Notizia	3
1.1.3	Lista dei preferiti	4
1.2	Codice	4
1.2.1	Utente	4
1.2.2	News	5
1.2.3	Favorite	6
1.2.4	Views & Services	6
1.2.5	Utilizzo dell'API Gemini	8
1.3	Sicurezza	14
1.3.1	Secure By Design	14
1.3.2	Implementazione - Criteri di Sicurezza	15
2	Scalabilità e processi asincroni	18
2.1	Esecuzione Disaccoppiata tramite Code di Task Asincrone	18
2.2	Resilienza del Sistema: Message Broker Dedicato	19
2.3	Portabilità, Coerenza e Containerizzazione	19
2.4	Gestione dei Processi e Scaling Indipendente dei Servizi	20
3	Frontend - Flutter	21
3.1	Framework Flutter	21
3.2	Funzionalità dell'applicativo	21
3.2.1	Layout e Internazionalizzazione	21
3.2.2	Login e Registrazione	23
3.2.3	Home hub & Funzioni	24
3.2.4	Discover Page	25
3.2.5	Favorite Page	25
3.3	Salvati	26
3.3.1	Profile Page	27
3.4	Impostazioni	27
3.5	Notizia - Rappresentazione grafica	28
3.6	Struttura del progetto frontend e integrazione dell'API (Django Rest Framework)	30

1 Backend - Django

1.1 Modello Entità Relazione

Al fine di gestire al meglio le entità presenti nel backend è stata eseguita, come di consueto, una fase di progettazione preliminare del DBSM. Le tabelle presenti nel backend sono descritte dal modello E/R in figura (fig.1).

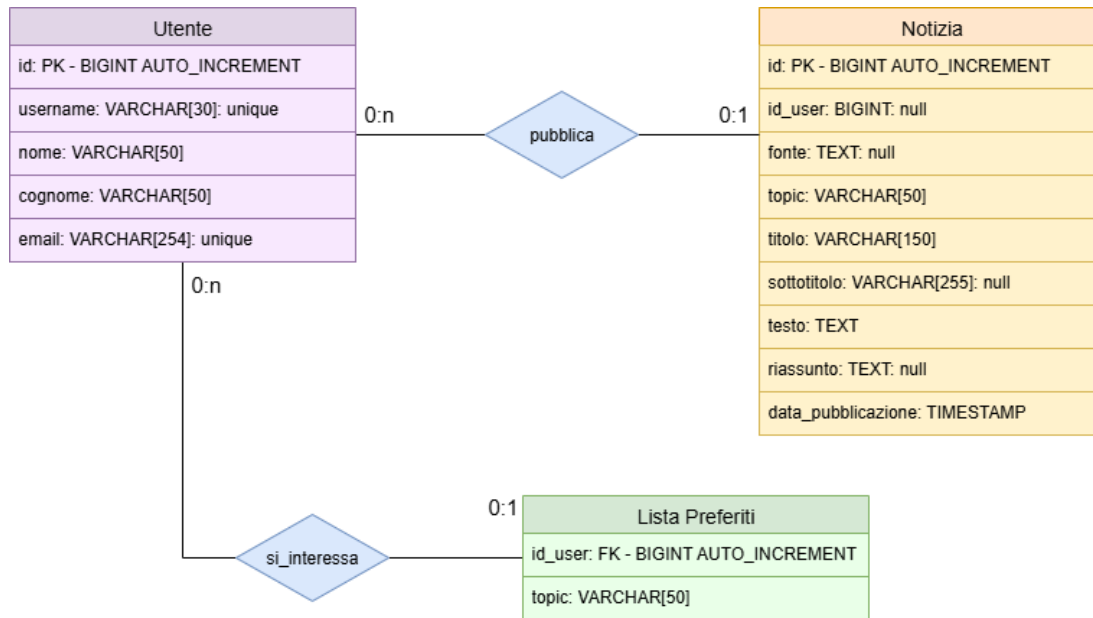


Figura 1: Schema "Entità Relazione" del DB

Lo schema risulta essere abbastanza autoesplicativo, tuttavia, per maggiore chiarezza analizzeremo nello specifico le varie entità.

1.1.1 Utente

Vista la tipologia di applicativo, non si è ritenuto necessario richiedere dati sensibili all'utente; gli unici dati che l'utente deve fornire sono: username (univoco), nome, cognome ed email (univoca). Ogni utente potrà essere riconosciuto a partire dal suo username o dalla sua email; l'id, invece, viene automaticamente gestito da Django in fase di creazione dell'entità. Per garantire la coerenza dei dati, sono stati implementati dei validatori al fine di processare l'input prima di accettarlo come valido; inoltre, non essendo stato implementato alcun tipo di verifica dell'identità, per i campi "nome" e "cognome" bisognerà fidarsi dell'input fornito dall'utente.

1.1.2 Notizia

Ogni notizia mantiene al suo interno più informazioni: id utente, fonte, topic, titolo, riassunto, sottotitolo, testo, eventuale generazione con l'ausilio dell'IA e data di pubblicazione; anche in questo caso l'identificativo della notizia viene automaticamente gestito da Django in fase di creazione.

Come specificato dalle linee guida per l'implementazione del progetto, tra le varie features, l'utente potrà caricare delle notizie da lui redatte. A tal proposito, si è pensato di implementare dei "meccanismi intelligenti" per la verifica del contenuto delle notizie postate; in altre parole, si sfrutta l'API di Gemini in maniera tale che agisca come un agente intelligente per la moderazione dei contenuti. Inoltre, al fine di gestire al meglio la sommarizzazione delle notizie e di prevenire lo spreco di risorse computazionali, alla prima richiesta viene generato e salvato un riassunto così da garantire una risposta tempestiva alle richieste future (meccanismo di caching). Ciò che avviene quando si richiede la sommarizzazione di una notizia consiste nella ricerca di un sunto pre-esistente, nel caso in cui questo dovesse mancare ne verrà generato e salvato uno.

L'applicativo è predisposto per reperire anche informazioni provenienti dal web, nel farlo, si baserà su una politica "Trusted Origins", ovvero si assume che tutte le informazioni reperite siano **affidabili** se e solo se lo sono anche le fonti dalle quali è stata recuperata l'informazione. L'API coinvolta nella creazione automatica delle notizie sportive è NEWS API¹.

Nel caso di una notizia reperita dal web, essa non conterrà alcun riferimento all'utente publisher², ma verrà mostrata la fonte dalla quale la notizia è stata reperita.

1.1.3 Lista dei preferiti

La lista dei preferiti è stata implementata come un'entità a sè; la sua funzione è quella di permettere all'utente di salvare un numero indefinito di "topic" di interesse al fine di garantire personalizzazione a livello UI/UX.

Il topic presente in "Notizia" e in "Lista dei preferiti" è identico e va interpretato come l'hashtag di un social network; di fatto ha la funzione di catalogare ogni notizia in base al contenuto³.

1.2 Codice

In questa sezione verranno riportati alcuni snippet di codice presi direttamente dall'implementazione del backend.

1.2.1 Utente

A livello di codice l'entità "utente" non viene dichiarata in maniera esplicita, di fatto, viene sfruttato il modello pre-esistente di Django che contiene tutte le informazioni necessarie. Infatti, nel codice è spesso presente la riga:

```
1      User = get_user_model()
```

Tale funzione, appartenente al package "Django.contrib.auth", restituisce il modello specifico di Django che rappresenta l'entità "Utente".

¹<https://newsapi.org/>

²In questo specifico caso non sarebbe presente un utente "publisher" e perciò non è necessario specificarne l'identificativo.

³Gli hashtag verranno applicati dall'utente in fase di pubblicazione e, successivamente, ne verranno aggiunti altri sfruttando strumenti IA, i quali garantiscono una corretta catalogazione anche per le notizie provenienti dal web.

1.2.2 News

La classe "News" è definita nel seguente modo⁴:

```
1      class News(models.Model):
2          user_id = models.IntegerField(...)
3          source = models.URLField(...)
4          topic = models.CharField(...)
5          title = models.CharField(...)
6          subtitle = models.CharField(...)
7          text = models.TextField(...)
8          summary = models.TextField(...)
9          publication_date = models.DateTimeField(...)
```

Analizzando i campi, uno per volta, troviamo:

- Riga 2: il campo "user_id" corrisponde all'id dell'utente che ha pubblicato la notizia. Tale valore potrebbe anche non essere presente, in quanto, come citato in precedenza, l'applicativo è in grado di reperire delle notizie sportive effettuando delle chiamate (a cadenza oraria) all'API "newsapi".
- Riga 3: "source", tale valore si ricongiunge a quanto esposto nel punto precedente, infatti, qualora dovesse mancare l'utente "publisher", allora la notizia avrà come fonte il sito dal quale è stata reperita tramite l'API.
- Riga 4: "topic", tale valore funziona come una sorta di "hashtag" utilizzato per catalogare la notizia. Il "topic" può essere specificato direttamente dall'utente in fase di pubblicazione⁵, oppure, nel caso in cui la notizia sia stata reperita dal web, verrà compilato tramite l'aiuto e l'analisi di Gemini.
- Riga 5, 6: i campi "title" e "subtitle", rappresentano letteralmente, "titolo" e "sottotitolo" della notizia.
- Riga 7: il campo "text" rappresenta il contenuto della notizia. Fino ad ora, tutti i campi analizzati hanno dei validatori di input basilari (controllo su stringhe vuote e date valide), invece, questo parametro gode della presenza di un validatore "intelligente".
Infatti, a differenza dei campi analizzati in precedenza, viene valutato il contenuto della notizia tramite l'utilizzo dell'API Gemini: qualora quest'ultimo dovesse essere considerato discriminatorio, minatorio, incitante all'odio ecc., allora la notizia non passerebbe i controlli e verrebbe bloccato il flusso di creazione dell'entità.

⁴Al fine di evitare problemi di formattazione, verranno riportate sezioni di codice in maniera incompleta. Per ulteriori approfondimenti consultare il sorgente. È importante sottolineare che il codice è stato appositamente scritto in lingua inglese per garantire una leggibilità maggiore e universale, lasciando aperti possibili sviluppi futuri.

⁵La scelta del campo "topic" non è "libera", bensì deve far parte di una lista di topic consentiti e reperibili all'url: <https://web-1083728320855.europe-west12.run.app/api/v1/topics/all/>

- Riga 8: il campo "summary" salva il riassunto inerente alla notizia. Lo scopo di tutto ciò è quello di emulare un meccanismo di caching per evitare di sprecare risorse computazionali. In altre parole, ogni volta che viene richiesto il riassunto di una notizia, se questo non dovesse essere già presente nel DB, allora ne verrà generato uno che sarà archiviato per gli utilizzi futuri.
- Riga 9: il campo "publication_date" rappresenta la data di pubblicazione della notizia che, viene reperito assieme alla notizia dal web, oppure impostato automaticamente in fase di pubblicazione.

1.2.3 Favorite

La classe "Favorite" si presenta in questo modo:

```
1 class Favorite(models.Model):
2     user = models.ForeignKey(...)
3     topic = models.CharField(...)
```

A differenza del modello "News", in questo caso non troviamo nessun campo "nullable" (non necessario), di fatto il topic deve necessariamente essere associato all'utente, altrimenti l'intera logica perderebbe di significato.

1.2.4 Views & Services

La documentazione degli end-point è facilmente reperibile al seguente url:

<https://web-1083728320855.europe-west12.run.app/api/schema/redoc/>

Tale documentazione viene compilata automaticamente a partire dal codice (compresi i relativi commenti) grazie all'integrazione della dipendenza "*redoc*" all'interno del backend.

Tuttavia, per semplicità, analizzeremo solo gli end-point di principale interesse. In questa sezione ci focalizzeremo su tre end-point:

1. news/<int:pk>/summarize/: end-point per effettuare la chiamata al servizio di sommarizzazione, il quale riassume la notizia con id "<int:pk>".
2. topics/all/: end-point per ricevere un dizionario contenente tutti i topic possibili.
3. add-to-favourite/: end-point tramite il quale l'utente può aggiungere uno specifico topic ai preferiti.

Sommarizzazione Il servizio di sommarizzazione implementato sfrutta l'API di Google Gemini. L'algoritmo inizia con la ricerca della notizia da riassumere, gestendo eventuali errori:

```
1 try:
2     news_article = News.objects.get(pk=pk)
```

Qualora dovesse essere già presente un riassunto per la notizia richiesta, allora questo verrebbe restituito in output senza effettuare calcoli aggiuntivi:

```
1     if news_article.summary:
2         return Response(
3             {"summary": news_article.summary},
4             status=status.HTTP_200_OK
5         )
```

In caso contrario, verrà sfruttata l'API di Google Gemini per generarne uno nuovo:

```
1     summary_text = generate_summary_with_gemini(news_article.text)
2
3     if summary_text:
4         news_article.summary = summary_text
5         news_article.save()
6
7         return Response(
8             {"summary": summary_text},
9             status=status.HTTP_200_OK
10        )
```

Topic consentiti Il seguente end-point è stato implementato per evitare errori di catalogazione, verificabili in casi in cui sussistano incongruenze tra gli input dell'utilizzatore e la logica di codifica.

```
1     class GetAllTopics(APIView):
2         permission_classes = [permissions.AllowAny]
3
4     def get(self, request):
5         return Response(
6             {"topics": topics},
7             status=status.HTTP_200_OK
8         )
```

È possibile osservare il tipo di responso alla quinta riga del blocco di codice, le categorie consentite e restituite dal codice (riga 6), sono elencate nella sezione "Appendice - Topic consentiti".

Aggiunta ai preferiti L'aggiunta ai preferiti funziona in maniera estremamente semplice: consiste in una semplice istanziazione di un oggetto "Favorite" con i campi "user_id" e "topic". La vera e propria lista dei preferiti corrisponde ad una query che restituisce tutti gli oggetti "Favorite" legati ad uno specifico utente.

```
1     class AddToFavourites(APIView):
2         permission_classes = [permissions.IsAuthenticated]
3
4     def post(self, request):
5         try:
6             user = request.user
7             topic = request.data['topic']
8             if not topic:
9                 return Response(
10                    {"error": "..."},
11                    status.HTTP_400_BAD_REQUEST
12                )
13            else:
14                if Favorite.objects.filter(
15                    user_id=user.id, topic=topic).exists():
16                    return Response(
17                        {"error": f"..."},
18                        status.HTTP_409_CONFLICT
19                    )
20                else:
21                    favorite = Favorite.objects.create(
22                        user_id=user.id, topic=topic)
23                    return Response(
24                        {"favorite": favorite},
25                        status.HTTP_201_CREATED
26                    )
27        except Exception as e:
28            return Response(
29                {"error": str(e)},
30                status.HTTP_500_INTERNAL_SERVER_ERROR
31            )
```

1.2.5 Utilizzo dell'API Gemini

Nella presente sezione verrà discussa la gestione dell'API di Google Gemini massivamente utilizzata nel codice per svolgere "task" di vario tipo. Per garantire leggibilità e modularità del codice, si è pensato di confinare tutte le funzioni AI all'interno di un unico file: "gemini_service.py".

Codice. Di seguito verrà riportato, per sommi capi, il codice presente nel file:

```
1  #Summaries:
2  def generate_summary_with_gemini(article_text: str)
3      -> str | None:
4
5      if not settings.GOOGLE_AI_STUDIO_API_KEY:
6          return None
7
8      model = genai.GenerativeModel(GEMINI_MODEL)
9
10     prompt = (...)
11
12     try:
13         response = model.generate_content(prompt)
14         return response.text
15     except Exception as e:
16         print(f"Error:...")
17         return None
18
19 #Moderator:
20 def check_content_for_offensiveness(
21     article_text: str) -> dict:
22
23     if not settings.GOOGLE_AI_STUDIO_API_KEY:
24         return {"is_offensive": False, "reason": "..."}
25
26     model = genai.GenerativeModel(GEMINI_MODEL)
27
28     prompt = (...)
29
30     try:
31         response = model.generate_content(prompt)
32         cleaned_response = response.text.strip()
33             .replace(''''json", ""').replace(''''', "")
34             .strip()
35
36         result = json.loads(cleaned_response)
37         return result
38     except (json.JSONDecodeError, Exception) as e:
39         print(f"Error:...")
40         return {"is_offensive": True, "reason": "..."}
41
42 #Cataloguer:
43 def catalog_article_with_gemini(
44     title: str, description: str) -> dict | None:
45
46     if not settings.GOOGLE_AI_STUDIO_API_KEY:
47         return None
```

```

48
49     model = genai.GenerativeModel(GEMINI_MODEL)
50
51     valid_topics_str = ",_".join(f'"{topic}"',
52                                   for topic in VALID_TOPICS)
53
54     prompt = (...)
55
56     try:
57         response = model.generate_content(prompt)
58
59         match = re.search(r'\{.*\}', response.text, re.DOTALL)
60
61         if match:
62             json_string = match.group(0)
63             result = json.loads(json_string)
64             return result
65         else:
66             print("Error:...")
67             return None
68
69     except json.JSONDecodeError as e:
70         return None
71     except Exception as e:
72         return None

```

Riassunti (riga 2) Tale funzione viene utilizzata per riassumere il testo delle notizie passate in input. Sono stati utilizzati dei prompt progettati appositamente per modellare l'output così che risulti compatibile con le esigenze implementative. Il prompt utilizzato per effettuare la sommarizzazione del codice è il seguente:

*"Sei un esperto giornalista sportivo. Riassumi il seguente articolo di cronaca sportiva in 3-4 frasi chiave, mantenendo un tono professionale e informativo. Continua a lavorare finché il lavoro non è completamente svolto prima di finire il tuo turno. Se non sei sicuro dei file o degli elementi allegati, aprili e non allucinare. Pianifica attentamente prima di ogni chiamata e rifletti sul risultato dopo. Ecco l'articolo:
testo dell'articolo..."*

Moderatore (riga 20) La funzione in analisi viene utilizzata come moderatore intelligente; venendo richiamato dal validatore utilizzato all'interno del modello "News" (sezione 1.2.2) in corrispondenza del campo "text" (riga 7). Tale moderatore si occupa di rilevare la presenza di contenuti offensivi, minatori, discriminatori ecc. all'interno del testo della nota. Il prompt che verrà dato in pasto a Gemini è il seguente:

"Sei un moderatore di contenuti esperto e molto rigido. Il tuo compito è analizzare il testo seguente per identificare se contiene linguaggio offensivo, incitamento all'odio, violenza esplicita, profanità eccessiva o attacchi personali.

Sii molto severo nel tuo giudizio. Ti prego di rispondere SOLO con un oggetto JSON con questa struttura:

```
{
  "is_offensive": true/false,
  "reason": "Una breve spiegazione del perché è offensivo
(es. Incitamento all'odio, Linguaggio volgare, Attacco personale)
oppure Nessuna violazione rilevata."
}
```

Se non dovessi rispettare la mia richiesta l'algoritmo di verifica fallirebbe e, a causa di ciò, molte persone potrebbero offendersi pesantemente.

Ecco il testo da analizzare:

testo dell'articolo..."

Nonostante non possa provare dei "sentimenti", il prompt riportato mira a chiarire il più possibile lo scopo dell'impiego dell'IA, provando a trasmettere una sorta di senso di responsabilità nei confronti degli utilizzatori.

Catalogatore (riga 43) L'algoritmo "catalog_article_with_gemini" viene utilizzato per catalogare gli articoli che il codice reperisce (ogni ora) tramite la chiamata all'API "newsapi.org". Per "catalogare" si intende: adattare e costruire un oggetto News in modo che sia conforme alle specifiche richieste dal sistema backend ("News" 1.2.2).

Il prompt utilizzato per fare ciò è il seguente:

"Sei un archivista di notizie sportive estremamente preciso e rigido. Analizza il titolo e la descrizione seguenti. Il tuo compito ha due passaggi OBBLIGATORI:

- 1. ****Verifica della Pertinenza:**** Per prima cosa, determina se il testo parla PRINCIPALMENTE di una competizione, un evento o un atleta sportivo. Se parla di politica, cronaca, economia o altro, anche se menziona uno sport, NON è una notizia sportiva valida.*
- 2. ****Catalogazione (SOLO se la verifica è positiva):**** Se e solo se la notizia è sportiva, estrai e genera le seguenti informazioni:*
 - ****topic****: Scegli ESATTAMENTE UNA delle seguenti categorie: **topic consentiti...** Non usare altre parole. Se nessuno corrisponde perfettamente, usa 'Other Disciplines'.*
 - ****subtitle****: Genera un sottotitolo accattivante di una sola frase.*
 - ****summary****: Genera un riassunto dettagliato di 3-4 frasi.*

Ti prego di rispettare le mie richieste altrimenti sarà tutto un fallimento.

Rispondi ESCLUSIVAMENTE con un oggetto JSON. Se la notizia è sportiva, la struttura deve essere:

```
{
  "is_sport_news": true,
  "topic": "Una Categoria Dalla Lista",
  "subtitle": "Sottotitolo generato.",
  "summary": "Riassunto generato."
}
```

Se la notizia NON è sportiva:

```
{
  "is_sport_news": false
}
TITOLO: titolo
DESCRIZIONE: descrizione
```

L'intelligenza artificiale riuscirà a catalogare con successo le notizie che verranno sottomesse. Ciò permetterà di automatizzare il processo di archiviazione delle notizie provenienti da fonti esterne, popolando il database con notizie reperite dalle "Trusted Origins".

```
Starting the fetch process of sport's news...
WARNING: Test-mode. Process maximum 2 as settings requirements.
Process the article: Torino, Vlasic: "Zapata ci darà un aiuto grandissimo. Baroni ci riempie di fiducia"
Process the article: Carlo Nesti: "Perché gli imprenditori piemontesi non comprano il Toro di Cairo?"
❌Discarded article by Gemini (error or not sport): Carlo Nesti: "Perché gli imprenditori piemontesi non comprano il Toro di Cairo?"
✅Process complete. Saved 1 new article.
```

Figura 2: Esempio di auto-fetching delle notizie reperite dal web. Caso: una notizia scartata

```
Starting the fetch process of sport's news...
WARNING: Test-mode. Process maximum 2 as settings requirements.
Process the article: Cecon, vita da star e per la prima volta dopo quattro anni senza ori
Process the article: Kelly, Sara, Chiara: quando gli altri sport sono stati un aiuto per avvicinarsi alla meta
✅Process complete. Saved 2 new article.
```

Figura 3: Esempio di auto-fetching delle notizie reperite dal web. Caso: nessuna notizia scartata

La logica di fetching per le notizie sportive avviene all'interno di un file chiamato "fetch_sport_news.py". Al fine di effettuare una query efficace, in grado di reperire un numero più ampio possibile di notizie sportive viene utilizzato il seguente url:

```
1 sport_sources = [
2     "gazzetta.it",
3     "sport.sky.it",
4     "tuttosport.com",
5     "corrieredellosport.it",
6     "sportmediaset.mediaset.it",
7     "eurosport.it"
```

```

8         ]
9         sources_string = ",".join(sport_sources)
10
11         api_url = (
12             f"https://newsapi.org/v2/everything?"
13             f"domains={sources_string}&"
14             f"language=it&"
15             f"sortBy=publishedAt&"
16             f"apiKey={settings.NEWS_API_KEY}"
17         )

```

Il processo di salvataggio e creazione dell'oggetto si trova alla riga 84 del codice:

```

1     try:
2         News.objects.create(
3             source=source_url,
4             text=article.get('content') or description,
5             title=title,
6             topic=cataloged_data.get('topic', 'Unclassified'),
7             subtitle=cataloged_data.get('subtitle'),
8             summary=cataloged_data.get('summary'),
9             publication_date=parse_datetime(original_pub_date_str)
10                if original_pub_date_str else None
11        )

```

Inoltre, per evitare di esaurire le chiamate a disposizione nel piano gratuito di "newsapi.org" e non superare il limite di chiamate per minuto predisposte dal piano gratuito di Gemini, sono stati impostati un numero limite di articoli per chiamata e dei time-out; alla riga 43:

```

1     if settings.MAX_ARTICLES_TO_FETCH > 0:
2         articles = articles[:
3             settings.MAX_ARTICLES_TO_FETCH]

```

Il meccanismo mostrato permette di modificare il limite di articoli comodamente dal file principale "settings.py".

Lo scheduling del processo avviene in un apposito file "scheduler.py":

```

1     def fetch_news_job():
2         try:
3             call_command('fetch_sports_news')
4             except Exception as e:
5
6     def start():
7         scheduler = BackgroundScheduler()
8         scheduler.add_job(
9             fetch_news_job,
10            'interval',
11            hours=1,
12            id='fetch_news_job_001',
13            replace_existing=True
14        )

```

```
15         scheduler.start()
```

Lo scheduler viene avviato automaticamente all'avvio del server, nel file "apps.py":

```
1     class SmartIaSportNewsConfig(AppConfig):
2         default_auto_field = 'Django.db.models.BigAutoField'
3         name = 'Smart_IA_Sport_News'
4
5     def ready(self):
6         if os.environ.get('RUN_MAIN') == 'true':
7             from . import scheduler
8             scheduler.start()
```

1.3 Sicurezza

In questa sezione verranno analizzati tutti i criteri di sicurezza utilizzati in fase di implementazione. Procederemo in ordine: analizzando innanzitutto la sicurezza fornita dal framework di sviluppo, per poi passare ai criteri implementati attivamente nell'infrastruttura backend.

1.3.1 Secure By Design

Django offre un sistema di archiviazione delle password flessibile e usa PBKDF2 di default. L'attributo "password" di un oggetto User è una stringa in questo formato⁶:

$$< algorithm > \$ < iterations > \$ < salt > \$ < hash > \quad (1)$$

Questi sono i componenti utilizzati per memorizzare la password di un utente, separati dal simbolo del dollaro e costituiti da: l'algoritmo di hashing, il numero di iterazioni dell'algoritmo (o "*work factor*"), il salt casuale e l'hash della password risultante. L'algoritmo è uno dei numerosi algoritmi di hashing unidirezionale o di memorizzazione delle password che Django può utilizzare.

Le iterazioni descrivono il numero di volte in cui l'algoritmo viene eseguito sull'hash. Il salt è il "*seed*" casuale utilizzato e l'hash è il risultato della funzione unidirezionale.

Per impostazione predefinita, Django utilizza l'algoritmo PBKDF2 con un hash SHA256, un meccanismo di allungamento della password raccomandato dal NIST.

Ciò dovrebbe essere sufficiente per la maggior parte degli utenti: è abbastanza sicuro e richiede un'ingente quantità di tempo per essere violato.

Tuttavia, a seconda delle esigenze, è possibile scegliere tra diversi algoritmi o persino utilizzare algoritmi personalizzati per adattare il meccanismo di salvataggio alle specifiche richieste.

Django sceglie l'algoritmo da utilizzare consultando l'impostazione `PASSWORD_HASHERS`. Si tratta di un elenco di classi di algoritmi di hashing supportati dall'installazione di Django presente sulla macchina.

⁶Informazioni reperite dalla documentazione ufficiale di Django, disponibile all'URL: <https://docs.djangoproject.com/it/4.0/topics/auth/passwords/>

La prima voce di questo elenco (`settings.PASSWORD_HASHERS[0]`) verrà utilizzata per memorizzare le password, mentre tutte le altre voci sono hash validi che possono essere utilizzati per verificare le password esistenti.

Ciò significa che se si desidera utilizzare un algoritmo diverso, basterà modificare `PASSWORD_HASHERS` in modo che l'algoritmo desiderato sia il primo dell'elenco.

In conclusione, per quanto riguarda l'autenticazione e la *"Password Policy"*, sono stati attivati dei validatori standard (`UserAttributeSimilarityValidator`, `MinimumLengthValidator`, `CommonPasswordValidator`, `NumericPasswordValidator`) al fine di imporre una complessità minima delle password. Inoltre, `REST_FRAMEWORK` è configurato con `IsAdminUser` come permesso di default, seguendo il principio del *deny by default*: le API sono chiuse a meno che non vengano esplicitamente aperte.

1.3.2 Implementazione - Criteri di Sicurezza

L'architettura di sicurezza del backend è stata progettata seguendo il principio della *Defense in Depth*, implementando molteplici livelli di protezione che spaziano dalla gestione dei segreti infrastrutturali fino alla sicurezza a livello applicativo e di trasporto. La configurazione, definita nel file `settings.py`, adotta un approccio dinamico, distinguendo l'ambiente di sviluppo da quello di produzione per preservare la *developer experience*.

```
1     if DEBUG:
2         # Security OFF, Local-IP, ecc.
3
4     else:
5         # Security ON, Production-IP, ecc.
```

Nello specifico, il file `settings.py` integra una logica condizionale in grado di adattare automaticamente i parametri di sicurezza, facilitando, ove necessario, le attività di debug anche in fase post-deploy.

Gestione dei Segreti e delle Configurazioni Sensibili La gestione delle credenziali sensibili non è hardcoded nel codice sorgente, ma avviene dinamicamente tramite integrazione con **Google Cloud Secret Manager**. Il meccanismo alla base consiste in una logica di *"Recupero Dinamico"*, ovvero, all'avvio l'applicazione verifica la presenza della variabile d'ambiente `GOOGLE_CLOUD_PROJECT`. Se presente, istanzia un client `SecretManagerServiceClient` per recuperare il payload JSON contenente le variabili d'ambiente (inclusi `SECRET_KEY`, chiavi API e credenziali DB). In tal modo si riesce a centralizzare la gestione dei segreti e a prevenire l'esposizione di dati sensibili nel repository di codice (`git`). Inoltre sono presenti varie tipologie di *"Fail-safe Mechanism"*. Un esempio può essere il controllo bloccante sulla `SECRET_KEY`. Se questa non viene popolata (né da Secret Manager né dal file `.env`), l'applicazione solleva un `ValueError`, impedendo l'avvio del server in uno stato non sicuro e inconsistente.

Sicurezza del Trasporto (HTTPS e HSTS) In ambiente di produzione (`DEBUG = False`), vengono applicate policy stringenti per garantire la confidenzialità e l'integrità dei dati in transito.

La prima misura è l'*SSL Redirect*, implementato tramite la direttiva `SECURE_SSL_REDIRECT`. Tale configurazione impone l'instradamento di tutte le richieste HTTP non sicure verso il protocollo HTTPS.

Fondamentale è inoltre la configurazione del *Proxy Header Trust*. Tramite `SECURE_PROXY_SSL_HEADER`, l'applicazione viene istruita a riconoscere l'header `X-Forwarded-Proto` inviato dal reverse proxy (o load balancer), passaggio essenziale per gestire correttamente la terminazione SSL in ambienti containerizzati.

Infine, viene applicato il protocollo *HSTS* (*HTTP Strict Transport Security*) attraverso le seguenti impostazioni: `SECURE_HSTS_SECONDS = 31536000` (1 anno), `SECURE_HSTS_INCLUDE_SUBDOMAINS` e `SECURE_HSTS_PRELOAD`. Questa configurazione obbliga i browser a rifiutare categoricamente connessioni non cifrate per il dominio e i relativi sottodomini, mitigando efficacemente vettori di attacco quali *protocol downgrade* e *cookie hijacking*.

Protezione da Attacchi Web Comuni Il middleware di Django è stato configurato per mitigare le vulnerabilità critiche classificate nella *OWASP Top 10*, indirizzando specifici vettori di attacco quali *Host Header Injection*, *Cross-Site Scripting* (XSS) e *Cross-Site Request Forgery* (CSRF).

Relativamente alla *Host Header Validation*, la variabile `ALLOWED_HOSTS` viene valorizzata dinamicamente: ciò previene l'iniezione di header host arbitrari, garantendo che l'applicazione elabori esclusivamente le richieste indirizzate ai domini esplicitamente autorizzati.

Sul fronte della sicurezza client-side, la direttiva `SECURE_BROWSER_XSS_FILTER` abilita i meccanismi di filtraggio XSS nativi dei browser moderni. Contestualmente, `SECURE_CONTENT_TYPE_NOSNIFF` inibisce il *MIME sniffing*⁷, obbligando il client a rispettare rigorosamente il Content-Type dichiarato dal server e impedendo l'esecuzione di file mascherati.

Per la difesa contro attacchi di *UI redress* (come il *Clickjacking*), è attivo il `XFrameOptionsMiddleware`, che impedisce il rendering delle risorse all'interno di tag `<frame>` o `<iframe>`. Infine, la protezione *CSRF* viene rafforzata in produzione tramite `CSRF_COOKIE_SECURE`, parametro che vincola la trasmissione del token di sicurezza alle sole connessioni cifrate (HTTPS).

⁷L'acronimo MIME (*Multipurpose Internet Mail Extensions*) designa lo standard IETF utilizzato per la codifica e l'identificazione univoca della natura dei dati trasmessi su Internet (testo, audio, video, applicativi). La tipologia del contenuto, o *Media Type*, di norma viene dichiarata dal server all'interno dell'intestazione `HTTP Content-Type` delle risposte web.

Il *MIME sniffing*, invece, definisce il comportamento euristico adottato dai browser per dedurre il formato effettivo di una risorsa analizzandone la struttura dei byte (il cosiddetto *byte stream*), qualora il tipo non sia specificato o appaia incongruente. Durante questo processo, il browser ricerca firme digitali o pattern specifici e, in caso di corrispondenza, può decidere di ignorare il `Content-Type` dichiarato nell'intestazione HTTP, trattando la risorsa secondo la propria interpretazione.

Sebbene nato per garantire la compatibilità in presenza di configurazioni server errate, tale meccanismo introduce una superficie di attacco critica.

Sfruttando questa discrepanza, attori malevoli possono caricare file contenenti codice dannoso (es. script JavaScript) camuffati da formati innocui (es. immagini JPEG). Se il browser "sniffa" il contenuto ed esegue il file come codice anziché visualizzarlo come immagine, si concretizzano vulnerabilità quali *Cross-Site Scripting* (XSS), *code injection* e *information leakage*. Fonti: <https://www.keycdn.com/support/what-is-mime-sniffing> <https://stackoverflow.com/questions/18337630/what-is-x-content-type-options-nosniff>

Sicurezza dei Cookie e delle Sessioni Al fine di irrobustire la gestione delle sessioni utente sono stati impiegati specifici attributi dei cookie, la cui applicazione è modulata in base all'ambiente di esecuzione.

Il "*Secure Flag*", attivato in produzione tramite `SESSION_COOKIE_SECURE`, vincola la trasmissione dell'identificativo di sessione esclusivamente ai canali cifrati (HTTPS), prevenendo l'intercettazione dei cookie su reti non sicure (es. attacchi *Man-in-the-Middle (MIM)*).

Sul versante protocollare, "*HttpOnly*" viene implementato tramite `SESSION_COOKIE_HTTPONLY`, che inibisce l'accesso ai cookie da parte degli script lato client (es. `document.cookie`). Tale misura mitiga drasticamente il rischio di *Session Hijacking* e furto di credenziali anche qualora un attaccante riesca a sfruttare una vulnerabilità XSS.

Per mitigare le vulnerabilità ad attacchi di tipo Cross-Site Request Forgery si è optato per una politica *SameSite*, implementata tramite `SESSION_COOKIE_SAMESITE`, che stabilisce un compromesso ottimale tra sicurezza e *user experience*. Essa blocca l'invio del cookie nelle richieste *cross-site* (es. chiamate POST da domini esterni), proteggendo dai CSRF, pur garantendo la persistenza della sessione durante la navigazione ordinaria (le cosiddette *top-level navigations*).

CORS (Cross-Origin Resource Sharing) La politica CORS è gestita dinamicamente tramite la libreria `django-cors-headers`. La whitelist delle origini permesse (`CORS_ALLOWED_ORIGINS`) viene caricata dalle variabili d'ambiente, permettendo al backend di servire risorse solo a frontend fidati, bloccando richieste AJAX non autorizzate da domini terzi.

2 Scalabilità e processi asincroni

In questa sezione verranno illustrate le decisioni architetturali strategiche intraprese per garantire la scalabilità, la resilienza e l'efficienza operativa dell'infrastruttura backend del progetto.

In una prima fase di sviluppo si è optato per un modello di esecuzione sincrono e monolitico; invece, il modello scelto nella fase conclusiva dello sviluppo si presta a sostenere la possibile crescita futura, infatti, il sistema è in grado di gestire carichi di lavoro variabili e ottimizzare i costi operativi in un ambiente cloud-native.

In seguito verranno approfonditi i quattro pilastri fondamentali dell'architettura, ovvero la bufferizzazione dei task (promuovendone l'esecuzione asincrona), l'integrazione di un message broker dedicato, la containerizzazione dell'intera applicazione e la sua scomposizione in servizi autonomi, ciascuno gestito in modo indipendente per ottimizzare l'allocazione delle risorse e la manutenibilità.

2.1 Esecuzione Disaccoppiata tramite Code di Task Asincrone

Una delle principali sfide nell'implementazione dell'applicativo si è trovata nella gestione delle operazioni I/O-bound⁸.

Le chiamate alle API esterne di intelligenza artificiale, come quelle fornite da Google Gemini, sono un esempio calzante di questa problematica.

Utilizzando un paradigma sincrono, il processo del server web che gestisce la richiesta di un utente sarebbe costretto a rimanere in uno stato di attesa passiva fino al completamento della chiamata API.

Questo tipo di scelta rappresenterebbe un duplice errore, sprecando risorse computazionali e creando lunghe attese, degradando la User Experience (UX). In altre parole, la scelta di un paradigma sincrono corrisponde al bloccaggio di un intero processo, reso incapace di soddisfare altre richieste.

Al fine di superare tale limitazione, si è pensato di implementare un sistema di code di task distribuite basato su **Celery**. Aniché eseguire l'analisi del testo, la sommarizzazione (o tutti gli altri task a carico dell'API di Google Gemini) in tempo reale all'interno del ciclo di richiesta-risposta, il server web diventa "produttore" di task.

Al momento della sottomissione del compito asincrono, l'applicazione si limita a serializzare le informazioni necessarie e a registrarle come dei task in una coda persistente. L'utente riceverà immediatamente una risposta di conferma della presa in carico della richiesta specifica. L'esecuzione effettiva del compito è delegata a un pool di processi separati e indipendenti: i "worker" Celery.

⁸Con "operazioni I/O-bound" si fa riferimento a quei processi per i quali il tempo di esecuzione è determinato dall'attesa del completamento di operazioni di input/output. Nel caso specifico dell'implementazione in analisi, tali operazioni risultano lente e bloccanti, inducendo un degrado delle performance. Tale comportamento impatta negativamente la reattività dell'interfaccia, ostacolando un'interazione fluida e intuitiva da parte dell'utente. La caratteristica fondamentale di un'operazione I/O-bound è che la sua velocità non dipende dalla capacità di elaborazione del processore. Infatti, se il file in input è di grandi dimensioni, tali operazioni potrebbero rappresentare il "collo di bottiglia" dell'intero sistema.

L'adozione di questo modello è stata motivata dalla sua intrinseca scalabilità. Anche se il carico di richieste dovesse incrementare notevolmente, non sarebbe necessario potenziare l'istanza del server web, bensì basterebbe aumentare dinamicamente il numero di worker Celery in esecuzione.

Tale elasticità consente di adattare la potenza di calcolo in maniera quasi istantanea in risposta ai possibili picchi di carico, garantendo tempi di elaborazione costanti e ottimizzando i costi, in altre parole, si è ottenuto un modello di allocazione di risorse dinamico e adattivo.

2.2 Resilienza del Sistema: Message Broker Dedicato

Introducendo un modello Produttore-Consumatore (PC) si è ritenuta necessaria l'adozione di un intermediario robusto e affidabile in grado di gestire la comunicazione fra le due parti. La scelta di affidarsi a **Redis** come message broker è stata dettata dalla volontà di ridurre le responsabilità e le mansioni relegate all'applicazione. Redis funge da "cassetta della posta" intelligente per la gestione dei compiti generati da Celery. I compiti inviati da Django non vengono istantaneamente tradotti in un worker, ma vengono depositati in un buffer (gestito da Redis). I worker si connetteranno a Redis e preleveranno i compiti disponibili per l'elaborazione.

Se, per qualsiasi motivo, i processi worker dovessero essere temporaneamente non disponibili, i compiti inviati dal server web non andrebbero persi, ma rimarrebbero memorizzati nella coda di Redis, in attesa che i worker tornino operativi.

La dinamica di bufferizzazione adottata garantisce l'integrità dei dati e la continuità del servizio, aumentando notevolmente la fault tolerance del sistema. In un'ottica di produzione su Google Cloud, questo servizio sarà fornito da **Memorystore for Redis**; soluzione scalabile e ad alta affidabilità che astrae completamente la complessità della gestione dell'infrastruttura del broker.

2.3 Portabilità, Coerenza e Containerizzazione

Per garantire la coerenza tra gli ambienti di sviluppo, test e produzione si è scelto di adottare la "containerizzazione" come standard per l'intero applicativo backend, utilizzando **Docker**. È stato definito un **Dockerfile**, ovvero un documento di testo che codifica in modo dichiarativo tutte le dipendenze e le configurazioni necessarie per costruire l'immagine containerizzata eseguibile.

In altre parole, tale file agisce da "linea guida alla progettazione" di un artefatto immutabile che contiene il codice applicativo: il runtime Python, le librerie di sistema e tutte le dipendenze del progetto.

Il container avrà lo stesso comportamento del backend "puro" godendo di svariati vantaggi, quali l'isolamento e la portabilità. L'infrastruttura esecutiva sottostante non crea alcun tipo di problema⁹.

⁹Docker gode di tali vantaggi grazie alla sua architettura, che sfruttando la virtualizzazione a livello di sistema operativo riesce a superare in efficienza le tradizionali macchine virtuali. Il cuore dell'isolamento risiede in due funzionalità del kernel Linux: i Namespaces e i Control Groups (cgroups).

I Namespaces partizionano le risorse del kernel e forniscono ad ogni container una visione isolata e privata del sistema (PID, NET e MNT). Al contempo, i cgroups gestiscono l'uso delle risorse fisiche al fine di evitare impatti negativi sui container circostanti o sull'host.

2.4 Gestione dei Processi e Scaling Indipendente dei Servizi

L'architettura del sistema si compone di due tipologie di processi distinti e con diverse responsabilità:

- Servizio **web**: gestore dell'interazione con l'utente.
- Servizio **worker**: esecutore di task in background.

Gestione e scalabilità di tali servizi devono essere gestibili in maniera completamente indipendente. Per simulare un ambiente multi-processo in fase di sviluppo locale, è stato utilizzato **honcho** (gestore di processi che legge un file di configurazione: **Procfile** per avviare e supervisionare entrambi i servizi). Il tutto al fine di replicare fedelmente il comportamento dell'ambiente di produzione per testare in anticipo il sistema.

La piattaforma di produzione è **Google Cloud Run** (come richiesto dalla traccia). L'immagine utilizzata in fase di sviluppo verrà deployata sotto forma di due servizi Cloud Run distinti:

1. Servizio **web**: configurato per eseguire il comando del server Gunicorn ed essere esposto pubblicamente.
2. Servizio **worker**: utilizzerà la stessa immagine, ma eseguirà l'override del comando di avvio per eseguire il processo Celery (servizio che non necessiterà di un endpoint pubblico).

Il risultato sarà un sistema flessibile ed efficiente, il quale permetterebbe (se necessario) di configurare policy di auto-scaling diverse per ciascun servizio¹⁰, garantendo una corretta (e dinamica) gestione e allocazione delle risorse computazionali, risultando in grado di massimizzare le performance e minimizzare le spese operative.

Il vantaggio dell'architettura risiede nella leggerezza e rapidità dei container a dispetto delle macchine virtuali. La portabilità, in quanto qualità a sé stante, è una causa diretta del concetto di immagine immutabile. In altre parole, il Dockerfile produce un'immagine read-only e autosufficiente che contiene l'intera applicazione.

L'esecuzione di un container coincide con l'istanza di tale immagine, a cui viene aggiunto un sottile strato scrivibile per le modifiche runtime (lasciando comunque l'immagine sottostante invariata, garantendo che tutti i container avviati da una specifica immagine rimangano identici). L'immagine Docker prodotta corrisponde all'unica unità atomica che poi verrà deployata in produzione. Il risultato è la coerenza assoluta tra ciò che è stato testato e ciò che viene eseguito.

¹⁰Per fare un esempio, si potrebbero mantenere un numero minimo di istanze web sempre attive al fine di garantire bassa latenza, mentre i worker potrebbero scalare da zero ad n istanze nel caso in cui ci siano task presenti nel buffer.

3 Frontend - Flutter

3.1 Framework Flutter

Flutter è un framework open-source dedicato allo sviluppo frontend, creato e supportato da Google. Questo strumento permette la realizzazione di applicazioni native multiplatforma per ambienti mobile (iOS, Android), web, desktop (Windows, macOS, Linux) e dispositivi embedded, partendo da un'unica base di codice.

L'architettura interna, organizzata a livelli, si articola in tre componenti fondamentali: il Framework, l'Engine e l'Embedder. Il nucleo del sistema è costituito dall'Engine, responsabile del rendering grafico e del disegno dell'interfaccia utente. L'Embedder, specifico per ogni piattaforma, gestisce l'integrazione con il sistema operativo ospite, garantendo l'accesso ai servizi di rendering, agli input e alla gestione del ciclo di vita dell'applicazione.

La compilazione diretta delle applicazioni in codice macchina nativo assicura prestazioni elevate e tempi di avvio ridotti¹¹.

Motivazioni L'adozione di Flutter come framework di sviluppo è stata una scelta strategica che mira a garantire un'implementazione professionale ed efficiente di due applicazioni distinte: un'interfaccia web e un'applicazione mobile. Questa decisione consente di offrire un'esperienza utente fluida e reattiva, equiparabile a quella delle applicazioni native, ottimizzando significativamente i tempi e i costi di sviluppo.

3.2 Funzionalità dell'applicativo

Approccio alla progettazione Come si evince dalla sezione dedicata al Framework Flutter (2.1), la generazione del sito web e dell'applicazione mobile avviene a partire dal medesimo codice sorgente. Tale unificazione è resa possibile dalla compatibilità delle librerie adottate in fase di progettazione; infatti, nonostante la presenza di differenti embedder e sistemi di decodifica, è essenziale l'utilizzo di moduli trasversali compatibili con entrambi gli ambienti di sviluppo.

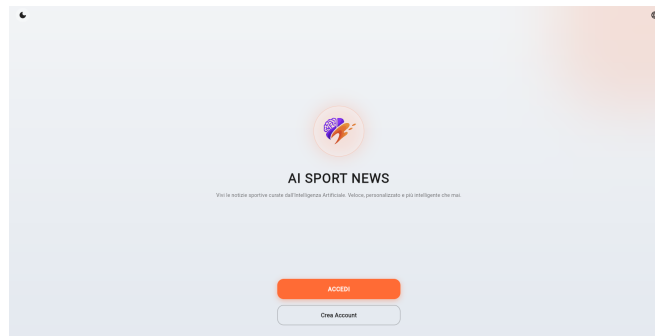
Ai fini di una maggiore chiarezza espositiva, nel prosieguo della trattazione si utilizzerà il termine "applicativo" per riferirsi indistintamente sia alla versione Web che a quella Mobile.

3.2.1 Layout e Internazionalizzazione

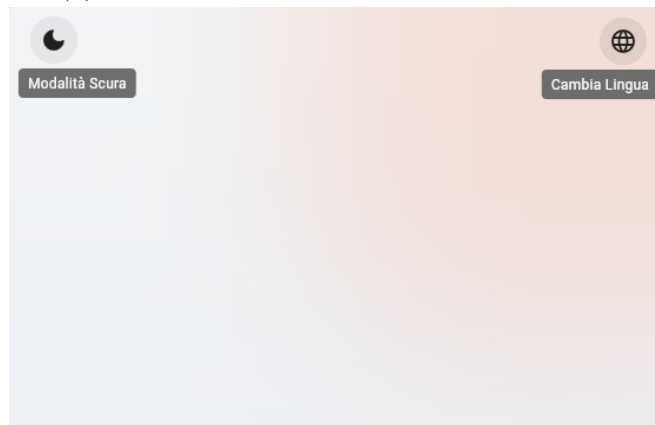
All'avvio, l'applicativo propone una schermata di benvenuto (fig. 4.a - 4.b) caratterizzata da widget UI essenziali ma funzionali, studiati per assicurare un'esperienza utente fluida ed efficace (fig. 4.c). In fase di progettazione si è posta particolare enfasi sull'internazionalizzazione e sulla manutenibilità del sistema. L'applicativo consente la commutazione dinamica della lingua dell'interfaccia; sebbene le notizie vengano visualizzate nella lingua originale di pubblicazione,

¹¹Una funzionalità distintiva in fase di sviluppo è l'Hot Reload stateful, che consente agli sviluppatori di visualizzare istantaneamente le modifiche al codice preservando lo stato corrente dell'applicazione.

l'architettura è stata strutturata per facilitare l'integrazione di nuovi idiomi e per predisporre il sistema all'eventuale traduzione automatica dei contenuti tramite API di terze parti. La scalabilità del supporto linguistico è garantita dall'organizzazione dei file di localizzazione (come `app_en.arb` e `app_it.arb`) all'interno di directory dedicate. Grazie a questa standardizzazione, l'aggiunta di una nuova lingua risulta un'operazione immediata, limitandosi alla redazione di un file di risorse strutturalmente identico a quelli preesistenti.



(a) Landing Page in inglese con tema chiaro

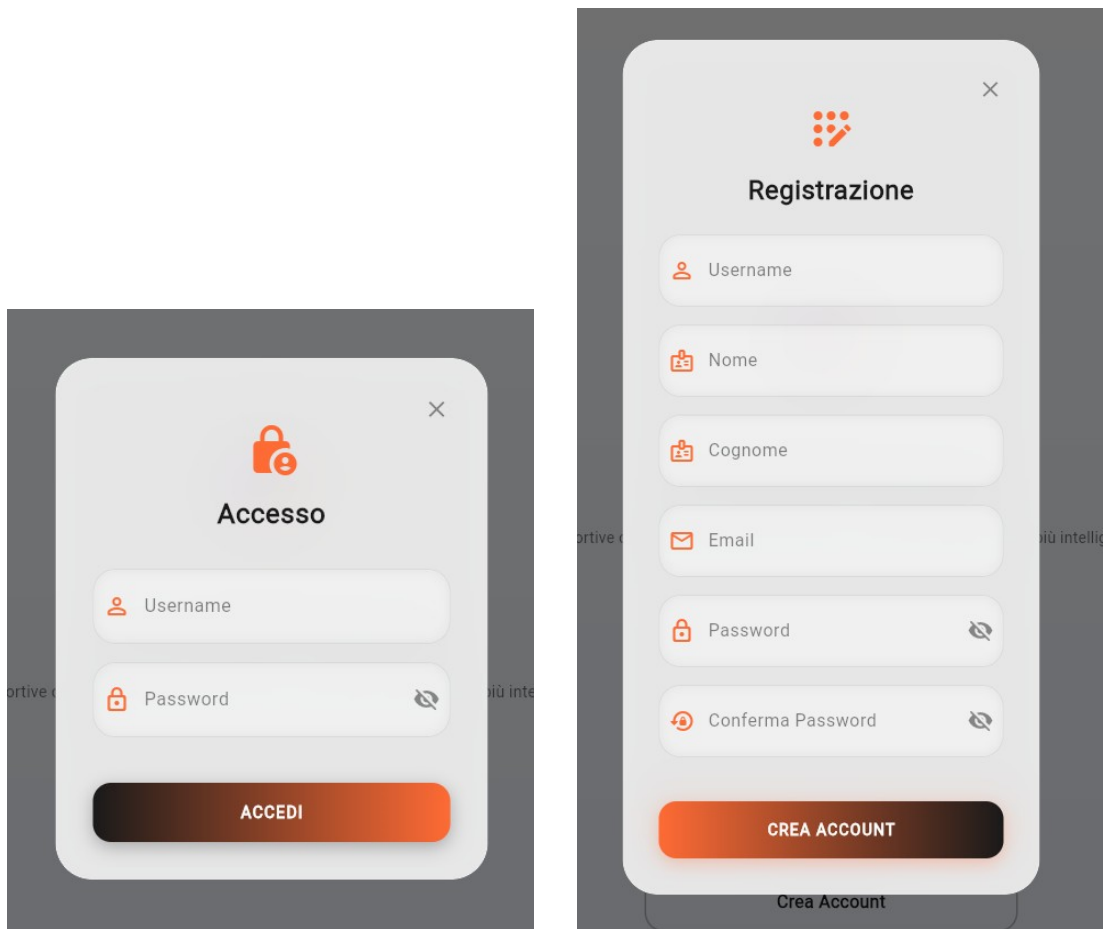


(b) UI della "Landing Page", permette di passare facilmente da un tema/lingua ad un altro/a

Figura 4: Landing Page.

3.2.2 Login e Registrazione

La Landing Page presenta due bottoni che permettono all'utente di entrare nel flusso di autenticazione.



(a) Popup di login

(b) Popup di registrazione

Figura 5: Popup di autenticazione: Pagina di Login (sinistra) e Pagina di Registrazione (destra)

Entrambe le funzioni utilizzano i servizi nativi previsti da Django allauth per l'autenticazione dell'utente.

3.2.3 Home hub & Funzioni

Home Page Subito dopo aver completato il processo di autenticazione, si arriva nella schermata home.

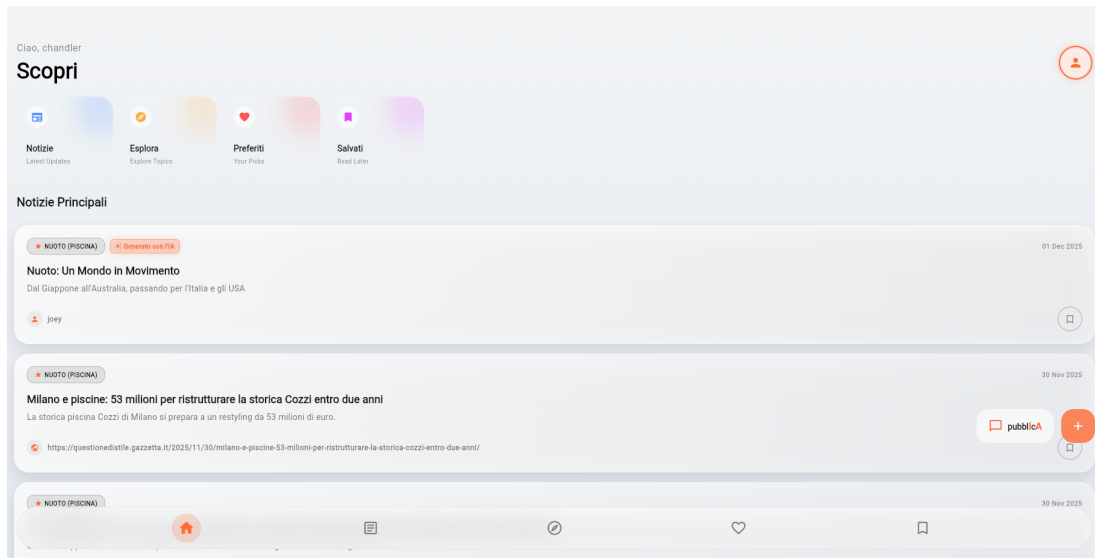


Figura 6: Hub Home Page & Notizie Recenti

All'interno della Home Page, l'utente ha la possibilità di consultare agevolmente le notizie più recenti. Come illustrato nella figura 6, la presenza del simbolo a forma di stella permette di distinguere visivamente gli articoli salvati come "preferiti" dal resto del flusso informativo. L'ordinamento delle notizie nella schermata principale risponde a due criteri gerarchici:

1. **Criterio principale:** basato sulla tematica. Le notizie afferenti agli argomenti indicati come preferiti dall'utente ottengono priorità di visualizzazione.
2. **Criterio secondario:** cronologico. Vengono mostrate esclusivamente le notizie pubblicate in un intervallo temporale non superiore ai tre giorni antecedenti la data odierna.

News Page L'accesso alla sezione "News" avviene mediante la barra di navigazione o l'hub dedicato. In questa pagina vengono visualizzate inizialmente quindici notizie; l'intero archivio rimane comunque consultabile avvalendosi della funzione di ricerca testuale o dei filtri avanzati.

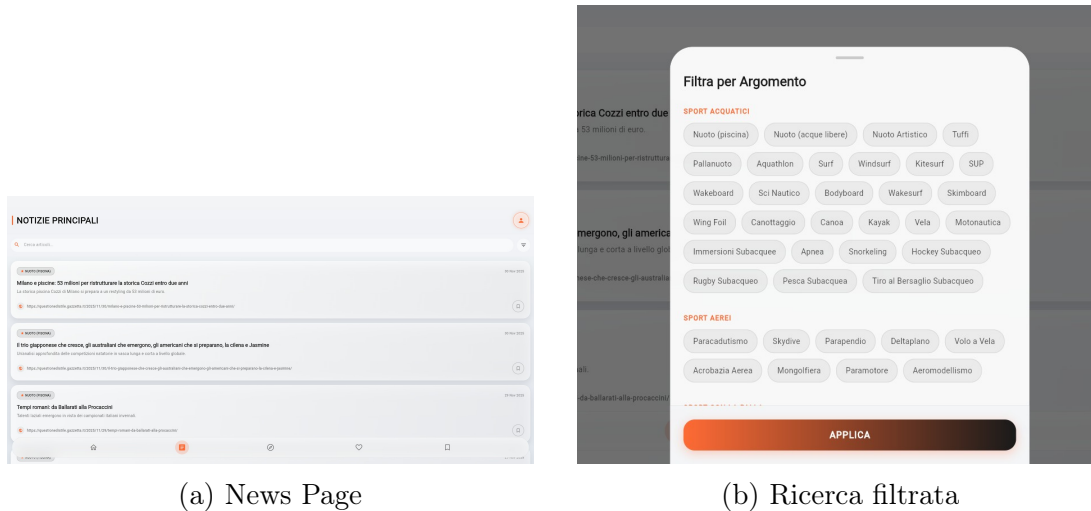


Figura 7: Funzionalità News Page

3.2.4 Discover Page

La pagina "Discover" è strutturata in due sezioni funzionali distinte: la prima è dedicata alla ricerca degli utenti registrati sulla piattaforma, mentre la seconda consente la consultazione dell'intero catalogo delle tematiche disponibili.

User Questa sottosezione permette di navigare tra i profili degli utenti registrati. Seguendo un paradigma simile a quello dei social network, il sistema consente di visualizzare lo storico delle notizie pubblicate da uno specifico utente, facilitando la fruizione dei contenuti generati dalla community.

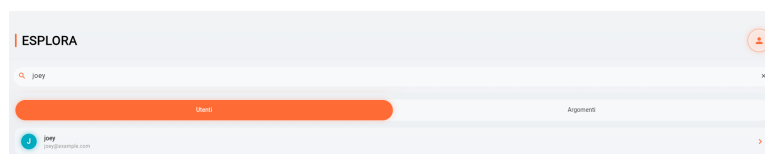


Figura 8: Interfaccia di ricerca utenti

Topics In quest'area è possibile esplorare la totalità degli argomenti trattati (topic). L'interfaccia offre la possibilità di gestire le proprie preferenze, permettendo all'utente di aggiungere un argomento alla lista dei preferiti interagendo con l'icona a forma di cuore.

3.2.5 Favorite Page

Questa schermata consente all'utente di consultare l'elenco degli argomenti (o "Topic") contrassegnati come preferiti, identificati visivamente dall'icona a for-

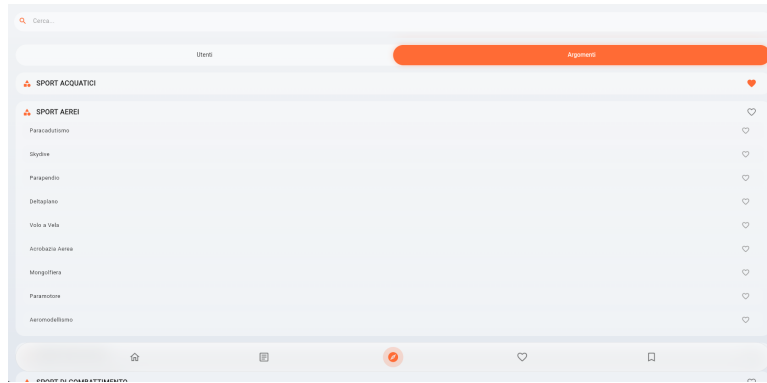


Figura 9: Visualizzazione e selezione dei topic

ma di cuore di colore giallo. Attraverso questa interfaccia, l'utente ha il pieno controllo sulla gestione dei propri interessi, con la facoltà di aggiungere nuove tematiche o rimuovere quelle non più rilevanti.

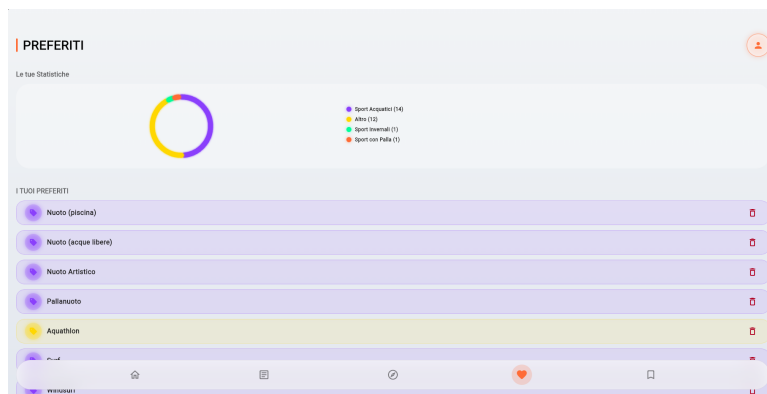


Figura 10: Schermata dei Topic preferiti

3.3 Salvati

Questa sezione funge da archivio personale: l'utente può leggere i post precedentemente salvati e, all'occorrenza, rimuoverli dalla lista dei contenuti memorizzati.

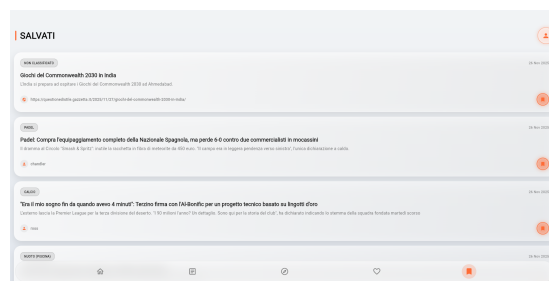


Figura 11: Archivio dei post salvati

3.3.1 Profile Page

L'area del profilo è dedicata alla gestione dell'account. In questa schermata è possibile modificare le informazioni personali, come lo username, e accedere al pannello di controllo delle notizie pubblicate, supervisionandone lo stato o procedendo alla loro rimozione.

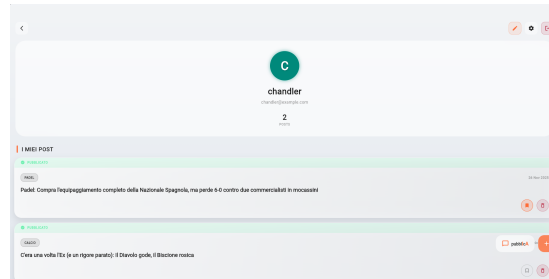


Figura 12: Gestione profilo utente

3.4 Impostazioni

Il pannello delle impostazioni consente la personalizzazione dell'esperienza d'uso, offrendo opzioni per la modifica della lingua e del tema grafico. L'essenzialità di questa sezione risponde a una precisa scelta progettuale, ispirata al principio di minimizzazione dei dati: si è evitato di raccogliere informazioni utente non strettamente funzionali all'utilizzo pratico dell'applicativo. Inoltre, un livello di complessità maggiore non era richiesto dagli obiettivi didattici del progetto.

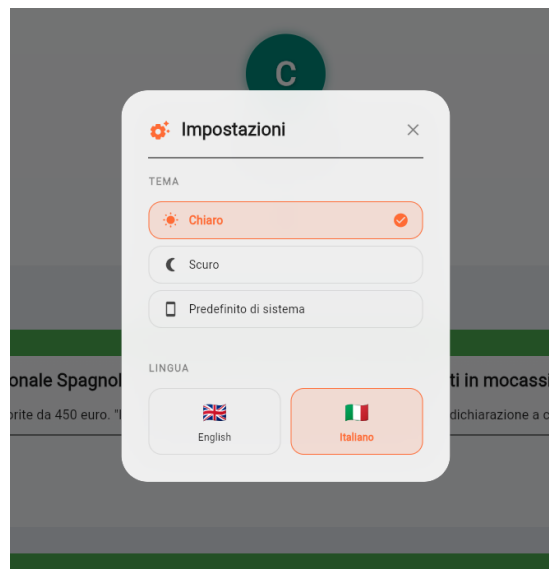
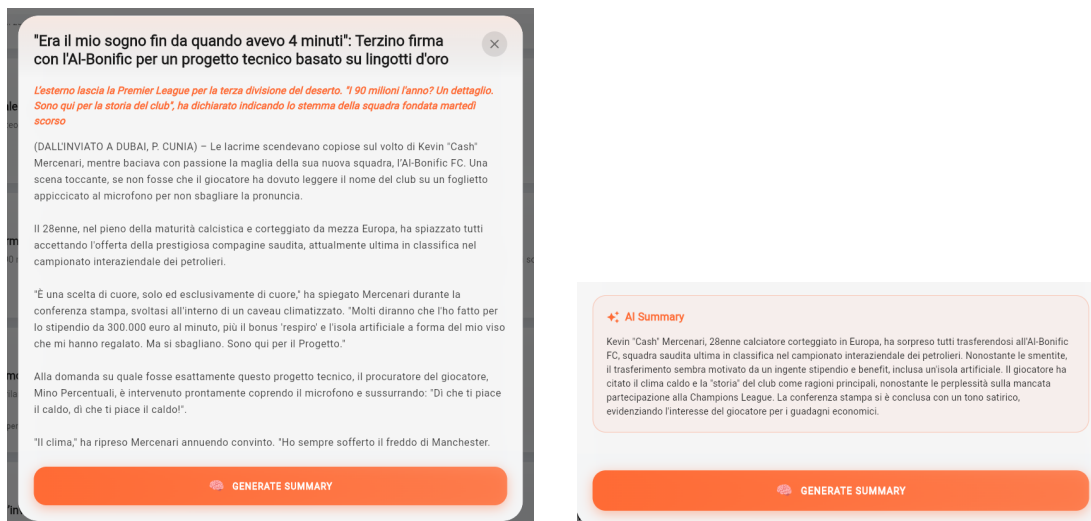


Figura 13: Caption

3.5 Notizia - Rappresentazione grafica

L'articolo viene presentato all'interno di un'interfaccia di lettura dedicata, dotata di una funzionalità per la generazione automatica di riassunti mediante intelligenza artificiale. Tale strumento è concepito per ottimizzare il processo di consultazione, agevolando la fruizione di testi particolarmente estesi e migliorando l'esperienza utente complessiva.



(a) Schermata di lettura

(b) Riassunto generato dall'IA

Figura 14: News popup reader e funzionalità AI

Si osserva che il corpo del testo è privo di formattazione HTML complessa o di elementi multimediali integrati. Tale limitazione deriva dalla natura delle API impiegate per il reperimento dei contenuti e l'elaborazione tramite Gemini; l'implementazione di una pipeline di rendering multimediale completo avrebbe comportato costi operativi non giustificati rispetto agli obiettivi prefissati per questo progetto. Tuttavia, un'eventuale evoluzione del sistema verso una gestione avanzata dei contenuti potrebbe essere attuata mediante l'adozione di Wagtail¹².

L'integrazione di tale CMS comporterebbe una ristrutturazione del modello dati: l'entità "Notizia" non sarebbe più definita come un semplice `models.Model`, bensì come una `Page`, tipologia di modello specifica dell'ecosistema Wagtail, previa configurazione degli URL e dei componenti necessari. La fase cruciale dell'implementazione risiede nella dichiarazione dei componenti strutturali dell'articolo. Di seguito viene proposta un'implementazione esemplificativa dei blocchi di contenuto all'interno di un ipotetico file `articles/blocks.py`.

¹²Wagtail è un CMS (Content Management System) basato su Django che consente di definire tipi di campo complessi denominati StreamFields. Uno StreamField permette all'utente di costruire l'articolo assemblando una serie di "blocchi" di contenuto in ordine arbitrario.

```

1 from wagtail import blocks
2 from wagtail.images.blocks import ImageChooserBlock
3 from wagtail.embeds.blocks import EmbedBlock
4
5 class ArticleTextBlock(blocks.StructBlock):
6     text = blocks.RichTextBlock(features=
7         ['h2', 'h3', 'bold', 'italic', 'link', 'ol',
8         'ul'], required=True)
9
10 class ArticleImageBlock(blocks.StructBlock):
11     image = ImageChooserBlock(required=True)
12     caption = blocks.CharBlock(required=False)
13
14 class ArticleQuoteBlock(blocks.StructBlock):
15     quote = blocks.TextBlock(required=True)
16     source = blocks.CharBlock(required=False)

```

Conseguentemente, il modello `News` verrebbe ridefinito ereditando dalla classe `Page`, sostituendo il campo testuale statico con uno `StreamField` dinamico che sfrutta i blocchi precedentemente dichiarati:

```

1 from django.db import models
2 from wagtail.models import Page
3 from wagtail import blocks
4 from wagtail.fields import StreamField
5 from wagtail.admin.panels import FieldPanel
6
7 from .blocks import ArticleTextBlock,
8     ArticleImageBlock, ArticleQuoteBlock
9
10 class NewsPage(Page):
11
12     sport = models.CharField(...)
13     subtitle = models.CharField(...)
14
15     content = StreamField([
16         ('heading', blocks.CharBlock(form_classname="title",
17         icon="title")),
18         ('paragraph', ArticleTextBlock(icon="pilcrow")),
19         ('image', ArticleImageBlock(icon="image")),
20         ('quote', ArticleQuoteBlock(icon="openquote")),
21         ('embed', blocks.RawHTMLBlock(icon="code")),
22     ], use_json_field=True, verbose_name="Contenuto □ Articolo")
23
24     cover_image = models.ForeignKey(
25         'wagtailimages.Image',
26         null=True,
27         blank=True,
28         on_delete=models.SET_NULL,
29         related_name='+',

```

```

30         verbose_name="Cover_Image",
31     )
32
33     content_panels = Page.content_panels + [
34         FieldPanel('sport'),
35         FieldPanel('subtitle'),
36         FieldPanel('cover_image'),
37         FieldPanel('content'),
38     ]
39
40     parent_page_types = ['HomePage']

```

L'adozione di `StreamField` offrirebbe significativi vantaggi tecnici, primariamente legati alla modularità dell'impaginazione. L'utente non sarebbe più vincolato a un unico blocco di testo monolitico, ma potrebbe comporre l'articolo attraverso una sequenza ordinabile di componenti discreti (testo, immagini, citazioni). Dal punto di vista dell'architettura dei dati, il contenuto verrebbe serializzato in formato JSON, garantendo una struttura pulita rispetto al salvataggio di stringhe HTML non strutturate. Infine, il rendering lato frontend avverrebbe iterando sui singoli blocchi, concedendo agli sviluppatori un controllo granulare sul markup e sullo stile di ogni componente. Questo approccio fornisce l'infrastruttura ideale per un sistema editoriale scalabile, adatto a gestire articoli sportivi complessi e ricchi di contenuti multimediali.

3.6 Struttura del progetto frontend e integrazione dell'API (Django Rest Framework)

Il codice sorgente è organizzato secondo l'architettura modulare tipica dei progetti sviluppati in Flutter, progettata per garantire la scalabilità e la manutenibilità del software multiplatforma. Di seguito viene riportata la struttura delle directory, seguita da un'analisi dettagliata dei moduli funzionali contenuti nella cartella principale `lib`.

La gerarchia delle cartelle evidenzia la separazione tra le configurazioni specifiche per piattaforma (Android, iOS, Web, Windows, macOS, Linux) e la business logic condivisa.

```

.
+---android
+---ios
+---linux
+---macos
+---web
+---windows
+---lib
    +---config
    +---l10n
    +---models
    +---pages
    |   +---auth

```

```
|   +---navigation
|   +---user_profile
+---services
+---theme
+---utils
+---widgets
+---test
```

Il nucleo dell'applicativo risiede nella directory `lib`, la quale è stata segmentata in sottocartelle tematiche per favorire la "Separation of Concerns" (SoC).

Configurazione e Internazionalizzazione La cartella `config` ospita i parametri globali dell'applicazione; in particolare, viene definito l'URL di base (*base URL*) per le chiamate API. Tale centralizzazione è strategica per facilitare la commutazione tra l'ambiente di sviluppo locale e l'ambiente di produzione (deployed) senza dover intervenire su porzioni di codice sparse. Parallelamente, il supporto multilingue è gestito all'interno della directory `l10n`, che contiene i file necessari per la localizzazione delle stringhe di testo, garantendo l'accessibilità dell'applicativo a un'utenza internazionale e semplificando possibili interventi futuri volti all'integrazione di altre lingue.

Layer dei Dati e Servizi La gestione dei dati è strutturata attraverso due componenti chiave: i Modelli e i Servizi. Nella cartella `models` risiedono i DTO (*Data Transfer Objects*), classi responsabili della mappatura delle entità ricevute dal database, quali utenti e notizie. L'interazione con il backend è invece delegata al modulo `services`, che centralizza le logiche di comunicazione. Qui troviamo servizi specifici come `auth_service` per l'autenticazione, `news_service` e `topic_service` per il recupero dei contenuti, e `chat_service` per l'interazione con le funzioni di intelligenza artificiale. Un ruolo cruciale è svolto dal `secure_storage_service`, incaricato della persistenza sicura dei cookie di sessione e dei token, garantendo la continuità dell'esperienza utente durante i vari accessi alla piattaforma.

Interfaccia Utente (UI) Il livello di presentazione è organizzato nelle cartelle `pages`, `widgets` e `theme`. La directory `pages` suddivide le schermate in base al flusso logico: la sezione `auth` gestisce il processo di ingresso (Landing page, Login e Registrazione), `navigation` raggruppa le schermate principali dell'applicativo, mentre `user_profile` è dedicata alla gestione dell'account utente. Per garantire coerenza grafica e riutilizzabilità del codice, i componenti visivi modulari sono raccolti in `widgets`, mentre la definizione stilistica globale, inclusa la gestione delle modalità chiaro/scuro, è confinata nella cartella `theme`.

Conclusioni

Il progetto "AI Sport News" ha permesso di approfondire e applicare concretamente i paradigmi fondamentali del Cloud Computing, dimostrando come la coordinazione di servizi eterogenei (Django, Celery, Redis, Google Cloud Platform, API Google Gemini, News API, ecc.) possa convergere in un'architettura scalabile, resiliente e moderna. L'applicativo realizzato risponde pienamente ai requisiti di modularità e sicurezza prefissati, offrendo un'esperienza utente fluida sia su piattaforme web che mobile grazie all'adozione del framework Flutter.

Tuttavia, il processo di progettazione è stato caratterizzato anche da precise scelte di design funzionale, volte a definire l'identità della piattaforma. In particolare, durante l'analisi dei requisiti per la componente frontend, si è valutata l'introduzione di meccanismi di interazione sociale, specificamente la funzionalità di "following" tra utenti. Nonostante l'interfaccia richiami esplicitamente il paradigma dei social network (*social media alike*), si è deciso intenzionalmente di non implementare tale meccanismo nella versione attuale. La decisione nasce dalla volontà di evitare l'instaurarsi di dinamiche di confronto sociale e di competizione tra gli utenti; l'obiettivo primario è stato quello di estirpare sul nascere una "meritocrazia basata sui follower", dove la visibilità dei contenuti è dettata dalla popolarità dell'autore, piuttosto che dall'effettiva qualità e veridicità della notizia sportiva riportata.

In ottica di sviluppi futuri, l'infrastruttura backend è già tecnicamente predisposta per accogliere funzionalità aggiuntive senza richiedere significativi refactoring strutturali:

- **Sistema di Notifiche Push:** Sebbene non implementato in questa release, il sistema di task asincroni basato su **Celery** e **Redis** costituisce già la fondazione necessaria. L'integrazione di **Django Beat** per lo scheduling periodico, in combinazione con servizi esterni come **Firestore Cloud Messaging (FCM)**, permetterebbe di gestire agevolmente l'invio di notifiche real-time agli utenti.
- **Autenticazione OAuth2:** L'accesso tramite account Google, pur non essendo stato integrato, risulta facilmente implementabile. Grazie alla struttura modulare del *Security Service* e alla gestione flessibile dei token nel backend, l'aggiunta di provider di identità federata rappresenterebbe un'estensione naturale del sistema di autenticazione attuale.

In conclusione, il sistema "AI Sport News" rappresenta un solido punto di partenza per una piattaforma di informazione sportiva collaborativa, tecnicamente robusta e progettata con un'attenzione consapevole alle dinamiche dell'esperienza utente.

Appendice

Topic consentiti

Di seguito riportata la lista dei topic consentiti:

• Sport Acquatici

- Nuoto (piscina)
- Nuoto (acque libere)
- Nuoto Artistico
- Tuffi
- Pallanuoto
- Aquathlon
- Surf
- Windsurf
- Kitesurf
- Stand Up Paddle (SUP)
- Wakeboard
- Sci Nautico
- Bodyboard
- Wakesurf
- Skimboard
- Wing Foil
- Canottaggio
- Canoa
- Kayak
- Vela
- Motonautica
- Immersione subacquea
- Apnea
- Snorkeling
- Hockey subacqueo
- Rugby subacqueo
- Pesca in apnea
- Tiro al bersaglio subacqueo

• Sport dell'Aria

- Paracadutismo
- Skydiving

- Parapendio
- Deltaplano
- Volo a vela
- Acrobazia aerea
- Mongolfiera
- Paramotore
- Aeromodellismo

• Sport con la Palla

- Calcio
- Calcio a 5 (Futsal)
- Beach Soccer
- Calcio gaelico
- Pallacanestro
- Pallamano
- Beach Handball
- Rugby
- Football americano
- Hockey
- Polo
- Lacrosse
- Pallavolo
- Beach Volley
- Tennis
- Badminton
- Tennistavolo
- Ping Pong
- Padel
- Pickleball
- Sepak Takraw
- Baseball
- Softball
- Cricket
- Golf
- Bocce

- Bowling
- Fistball
- Pelota basca

• Sport di Combattimento

- Pugilato
- Arti marziali miste (MMA)
- Lotta
- Judo
- Taekwondo
- Karate
- Kickboxing
- Muay Thai
- Jiu-Jitsu brasiliano
- Scherma
- Kendo
- Sumo
- Savate

• Atletica e Sport di Forza

- Corsa
- Salto in lungo
- Salto in alto
- Salto con l'asta
- Salto triplo
- Getto del peso
- Lancio del disco
- Lancio del martello
- Lancio del giavellotto
- Decathlon
- Eptathlon
- Sollevamento pesi
- Bodybuilding
- Powerlifting

• Ginnastica

- Ginnastica artistica
- Ginnastica ritmica

- Ginnastica con trampolino
- Ginnastica aerobica
- Ginnastica acrobatica

• Sport Invernali

- Sci alpino
- Discesa libera
- Super-G
- Slalom gigante
- Slalom
- Combinata
- Sci di fondo
- Salto con gli sci
- Combinata nordica
- Biathlon
- Sci freestyle
- Scialpinismo
- Snowboard
- Bob
- Slittino
- Skeleton
- Pattinaggio di velocità
- Pattinaggio di figura
- Short track
- Hockey su ghiaccio
- Curling
- Bandy

• Sport Motoristici

- Corse automobilistiche
- Formula 1
- Rally
- Corse di durata
- Corse turismo
- Karting
- Corse motociclistiche
- MotoGP
- Superbike
- Motocross

- Enduro
- Speedway
- Motonautica
- Corse aeree

- **Ciclismo**

- Ciclismo su strada
- Ciclismo su pista
- Mountain bike
- Ciclocross
- BMX
- Trial

- **Sport Equestri e con Animali**

- Equitazione
- Polo
- Ippica
- Endurance equestre
- Volteggio
- Sport cinofili

- **Sport di Precisione**

- Tiro a segno
- Tiro al piattello
- Tiro con l'arco
- Biliardo
- Bocce

- Bowling
- Curling
- Freccette
- Golf
- Footgolf

- **Sport della Mente**

- Scacchi
- Dama
- Go
- Bridge
- Poker
- Backgammon

- **Altre Discipline**

- Arrampicata sportiva
- Alpinismo
- Escursionismo / Trekking
- Orienteering
- Parkour
- Skateboard
- Pattinaggio a rotelle
- Danza sportiva
- Cheerleading
- Braccio di ferro
- Non classificato